Roteiro de Laboratório 2: Pipeline com vetores e medição de tempo

Unidade 04 | Continuação do Laboratório 1. Entrega pelo Moodle ao final da aula.

Objetivos

- Enviar vetores em vez de valores isolados, e receber mensagens cujo tamanho só é conhecido em tempo de execução.
- Fazer várias mensagens atravessarem o pipeline ao mesmo tempo, com os estágios trabalhando em paralelo.
- Medir tempo com `MPI_Wtime` de forma correta quando os processos estão em nós diferentes.

Preparação

Leve os arquivos de `codigo/` para a Atlântica como no Laboratório 1: colando com `cat > arquivo.c` ou pelo repositório do grupo no GitHub.

`pipeline.c` é a solução do Laboratório 1: um inteiro passa de estágio em estágio e cada estágio o incrementa. Se o seu ficou pronto, pode partir dele. Todas as partes deste roteiro modificam esse mesmo arquivo; guarde uma cópia ao fim de cada parte.

Parte 1: Vetores em vez de inteiros

1. Troque o inteiro por um vetor de `TAM` inteiros, alocado com `malloc` e com `TAM` lido da linha de comando (`./pipeline 1000`). O primeiro estágio preenche o vetor com zeros, cada estágio soma 1 a **todos** os elementos e o último confere se todos ficaram iguais ao número de estágios. Lembre de ajustar o segundo parâmetro de `MPI_Send` e `MPI_Recv`: ele é a quantidade de elementos, não de bytes.

2. Agora cada estágio intermediário passa a **descartar o último elemento** antes de enviar, como um estágio que filtra dados: o vetor encolhe ao longo do pipeline. O receptor não sabe mais quantos elementos vão chegar. Receba com o tamanho máximo (`TAM`) e descubra quantos chegaram de fato com:

   ```
   int recebidos;
   MPI_Get_count(&status, MPI_INT, &recebidos);
   ```

   Faça o último estágio imprimir quantos elementos recebeu.

3. **Responda:** o que acontece se o `MPI_Recv` for chamado com uma quantidade **menor** que a da mensagem que chega? E maior? Teste os dois casos.

Esse padrão (receber com folga e perguntar o tamanho ao `status`) é o que o modelo de divisão e conquista usa: cada processo recebe do pai um pedaço de tamanho variável.

Parte 2: Várias mensagens no pipeline

Com uma mensagem só, cada estágio trabalha uma vez e fica parado o resto do tempo. Um pipeline só rende quando várias mensagens estão em trânsito ao mesmo tempo.

1. Faça o primeiro estágio enviar `M` vetores em sequência, com `M` lido da linha de comando (`./pipeline 1000 100`). Cada estágio repete `M` vezes o ciclo receber, processar e enviar.

2. Para que o processamento tenha um custo mensurável, acrescente `usleep(10000)` (10 ms, de `<unistd.h>`) na etapa de processamento de cada estágio.

3. Rode no cluster com 8 estágios e `M = 1` e `M = 10`, e confira que todas as mensagens chegam ao último estágio.

Parte 3: Medir o tempo do pipeline

`MPI_Wtime()` devolve o tempo de parede em segundos, como `omp_get_wtime()`:

```
double t1 = MPI_Wtime();
/* ... */
double t2 = MPI_Wtime();
printf("Tempo: %f s\n", t2 - t1);
```

O objetivo é medir quanto tempo as `M` mensagens levam para atravessar o pipeline inteiro.

1. **Responda:** por que não basta o primeiro estágio ler `t1` antes de enviar, o último ler `t2` depois de receber e um deles calcular `t2 - t1`? Pense em onde cada processo roda quando `-N 2`.
2. Meça do jeito correto: todo o tempo é lido **no mesmo processo**. Chame `MPI_Barrier` antes de o processo 0 ler `t1`; depois de receber a última mensagem, o último estágio envia ao processo 0 uma mensagem curta de aviso; o processo 0 lê `t2` ao receber esse aviso e imprime o tempo.
3. Rode no cluster com 8 estágios e complete a tabela:

   ```
   srun -N 2 -n 8 ./pipeline 1000 1
   srun -N 2 -n 8 ./pipeline 1000 100
   ```

Mensagens (M)	Tempo medido (s)	Tempo se não houvesse sobreposição (s)
1		
10		
100		

   A última coluna é o tempo de processar as mensagens uma de cada vez, sem que uma mensagem entre no pipeline antes de a anterior sair: `M × 8 × 10 ms`.

4. **Responda:** o tempo com `M = 100` ficou perto de 100 vezes o tempo com `M = 1`? Mostre que ele se aproxima de `(P + M - 1) × 10 ms`, com `P` estágios, e explique de onde vem cada termo da fórmula.

Erros comuns

Sintoma	Causa provável
`MPI_ERR_TRUNCATE`	o `MPI_Recv` reservou menos elementos do que a mensagem tem
Último estágio confere valores errados	`MPI_Get_count` não foi usado e o laço percorre `TAM` elementos
O programa trava com `M > 1`	algum estágio repete o laço um número de vezes diferente dos outros
Tempo negativo ou absurdo	`t1` e `t2` lidos em processos diferentes

Entrega

- O `pipeline.c` final (vetores, `M` mensagens e medição de tempo).
- A tabela da Parte 3 e as respostas das três partes.